

ShopFlow

****Systeme de Gestion d'une Boutique en Ligne****

Mini-Projet Backend Spring Boot 3 / Frontend Next.js

****Enseignant : Inconnu****

Annee universitaire 2025 - 2026

Duree 3 semaines - Travail individuel

Étudiant

****yosrij****

Travail individuel

Stack

Java 21 - Spring Boot 3.4.13 - PostgreSQL 18 / H2 - Spring Security 6 + JWT 0.12.6

Springdoc 2.8.13 - MapStruct 1.5.5 - Next.js 16.3.5 + TypeScript + Tailwind 4

Depot

****unknown**** (backend 67 fichiers, frontend 11 routes)

Git: unknown - 3b7f7ba

Date

22 Septembre 2026

Soutenance 10 min (3 min demo + 3 min code + 4 min questions)

Rapport technique - 7 pages - Architecture, choix complexes, difficultes, repartition

Sommaire

1. Schema d'architecture technique (couches + flux) - p.2

2. Choix d'implementation les plus complexes - p.3-4

3. Difficultes rencontrees et solutions - p.5

4. Repartition des tâches - p.6

5. Grille d'evaluation & mapping endpoints - p.6

6. Installation & lancement - p.7

Annexe - Postman + Swagger - p.7

1 Schema d'architecture technique

Le projet respecte strictement l'architecture en couches imposee Spec 3.2 p.4 : Controller -> Service -> Repository -> Entity, avec DTO separes et Mapper. Aucune entite n'est exposee directement.

Flux de donnees - exemple creation commande

Client (Next.js) -> POST /api/orders avec Bearer token -> JwtAuthenticationFilter verifie signature + injecte SecurityContext -> OrderController @Valid -> OrderService @Transactional : verifie stock final (Product + Variant), decremente stock, calcule sousTotal/frais/totalTTC, applique Coupon, genere ORD-2026-XXXXX (@PrePersist), cree Order + OrderItems, vide Cart -> OrderRepository.save() -> Hibernate flush -> PostgreSQL (prod) / H2 (dev) -> retour OrderResponse DTO 201 + Swagger.

Frontend consomme via src/lib/api.ts : Axios + intercepteur injecte Bearer sur chaque requête, intercepte 401 -> POST /api/auth/refresh -> rejoue requête (Spec 4.3). Guards Next.js protegent /cart, /orders, /seller.

2 Choix d'implementation les plus complexes

2.1 Gestion du stock multi-niveaux (Product + Variant)

****Probleme**** : Spec 2.1 impose variantes taille/couleur avec stock individuel + verification temps reel panier/commande.
****Solution**** : Product.stock (global) + ProductVariant.stockSupplementaire + prixDelta. Dans CartService.addItem() et OrderService.createOrder(), stock disponible = product.stock + variant.stockSupplementaire. Verification a l'ajout panier ET a la creation commande (derniere verif p.3), decrement atomique en @Transactional. Si insuffisant -> 400 Stock insuffisant. Choix Hibernate LAZY + orphanRemoval pour eviter N+1.

2.2 Filtrage & recherche avec Specifications + Pageable

Spec 2.1 exige recherche plein texte nom/description/categorie/prix min/max + tri + pagination obligatoire. Au lieu de multiples methodes JPQL, ProductSpecifications.java expose 6 Specifications combinables : hasCategory, hasSeller, priceBetween, isPromo, isActif, search(q). Dans ProductService.getProducts() : Specification.where(isActif(true)).and(hasCategory(...)).and(priceBetween(...)) + productRepository.findAll(spec, pageable). Avantage : requête dynamique sans explosion de methodes, compatible JpaSpecificationExecutor, testable, performant (LAZY). Fallback JPQL search(q) et findTopSellingIds() via OrderItem GROUP BY.

2.3 Numero de commande unique & flux de statuts

Spec 2.4 : ORD-2024-XXXXX + PENDING -> PAID -> PROCESSING -> SHIPPED -> DELIVERED + annulation seulement si PENDING/PAID + remboursement. Implementation : Order.numeroCommande genere via @PrePersist generateNumero() = ORD-Year-5digits random + @Column(unique=true). Statut enum OrderStatus avec transition controlee dans OrderService.updateStatus() et cancel() (verifie if statut != PENDING && != PAID throw + restock product.stock += quantite pour remboursement simule). Chiffres logiques metier centralises en Service, pas en Controller.

2.4 Secure JWT stateless access + refresh

Spec 3.1 : Spring Security 6 + JWT jjwt access+refresh. Choix : jjwt 0.12.6 (patch CVE-2024-31033) + secret Base64 256 bits dans application.yml: app.jwt.secret (env JWT_SECRET en prod). JwtService genere access 1h + refresh 7j, signe HS384. JwtAuthenticationFilter (OncePerRequestFilter) extrait Bearer, valide signature + expiration, injecte UsernamePasswordAuthenticationToken. SecurityConfig stateless SessionCreationPolicy.STATELESS, permitAll sur /api/products/**, /api/categories/**, /auth/**, /swagger-ui/**, /h2-console/**, authenticated() ailleurs, DaoAuthenticationProvider + BCryptPasswordEncoder. Refresh : POST /api/auth/refresh {refreshToken} verifie validite puis re-emet access. Logout stateless -> 200. Frontend src/lib/api.ts intercepte 401 -> refresh -> rejoue requête (Spec 4.3).

2.5 Frontend Next.js vs React pur & TypeScript

Spec 4.1 laisse choix Next.js ou Angular. Choix Next.js 16.3.5 : framework React avec App Router (fichier -> route), SSR/SSG pour SEO catalogue (produits en vedette), optimisation images, server components. React seul aurait necessite react-router + config SSR manuelle. TypeScript obligatoire en pratique : types miroirs des DTO Spring Boot (Product, Cart, Order, Role) evitent erreurs prix: number vs string, autocompletion, garde-fous pour OrderStatusBadge, RatingStars composants reutilisables (Spec 4.3).

2.6 DTO separees + MapStruct (Spec 3.2)

Aucune entite exposee : ProductRequest (@NotBlank nom, @NotNull prix/stock) vs ProductResponse (+ sellerNom, categories Set, noteMoyenne, pourcentageRemise calcule). ProductMapper @Mapper(componentModel="spring") gere toEntity / toResponse + toCategoryDtoSet. @Data/@Builder evite boilerplate. GlobalExceptionHandler centralise 400 (@Valid) + 404.

2.7 Base double PostgreSQL/H2 + pg_hba

Spec 3.1 : PostgreSQL prod / H2 dev. application.yml avec 3 profils : active: dev par default -> jdbc:h2:mem:shopflow + ddl-auto: create-drop + H2 console enable. Profil prod -> jdbc:postgresql://localhost:5432/shopflow + HikariCP + dialect PostgreSQLDialect + env DB_USERNAME/PASSWORD. Passage en prod : DB_USERNAME=shopflow DB_PASSWORD=shopflow mvn spring-boot:run -Dspring-boot.run.profiles=prod. Probleme Fedora peer vs md5 resolu

(voir §3).

3 Difficultes rencontrees et solutions

****CVE Mend.io (logback 1.5.11, tomcat 10.1.31, js 2.17... 9.8)****

Probleme: Scanner Mend flaggeait 9.8 CVEs sur Spring Boot 3.3.5.

Solution: Upgrade 3.3.5 -> 3.4.13 + overrides postgresql 42.7.7, jackson 2.20.2, logback 1.5.18, commons-lang3 3.20.0 -> BUILD SUCCESS, restant 2026-* = false positives Insufficient Information documentes.

****pg_hba.conf Ident auth (Fedora 44)****

Probleme: FATAL: Ident authentication failed pour shopflow/localhost apres dnf install postgresql-server 18.

Solution: Passage host all all 127.0.0.1/32 md5 + local peer garde, systemctl restart postgresql, puis CREATE USER shopflow -> PGPASSWORD=shopflow psql -h localhost -> 1. pgAdmin 9.17 -> Host localhost (pas shopflow@localhost:5432).

****Java 25 par default vs 21 LTS****

Probleme: Fedora 44 installait OpenJDK 25 (too new pour Boot 3.4), mvn -version not found.

Solution: Utilisation du JDK IntelliJ ms-21.0.12.1 (unknown) + JAVA_HOME + Maven embarque Toolbox/.../maven3/bin/mvn. pom java.version 21.

****printf() redundant + typos****

Probleme: IntelliJ flaggeait printf("Hello") sans args + 10 typos francais dans pom.

Solution: Remplacement printf -> println + traduction commentaires pom en anglais, BUILD SUCCESS 67 files.

****Security 401 sur /api/products****

Probleme: Apres ajout Spring Security, GET /api/products -> 401 malgre permitAll initial.

Solution: Creation SecurityConfig avec SecurityFilterChain stateless, permitAll explicite sur products/categories/auth/swagger/h2, ajout JwtAuthenticationFilter avant UsernamePasswordAuthenticationFilter -> 200.

****H2 create-drop efface donnees****

Probleme: Chaque mvn spring-boot:run drop/recreate -> review test echouait (product not found).

Solution: Compris normal en dev (ddl-auto: create-drop); en soutenance utiliser prod ou update pour persistance. Data seeded via POST /api/categories + /products dans Postman.

4 Repartition des tâches

Travail individuel (yosrii) - 3 semaines decoupees selon ordre methodologique Spec p.7 :

Si binome : Backend (15 pts) : Spring Boot + JPA + Security (Dev A) ; Frontend (5 pts) : Next.js + Postman + Rapport (Dev B) - ici tout realise solo, choix justifie par stack homogene Java 21.

5 Grille d'evaluation & mapping endpoints

Mapping Spec p.5 -> implementation et points p.6 :

Endpoints couverts p.5: Auth (4), Products (6), Categories (4), Cart (6), Orders (6), Reviews (3), Coupons (4), Dashboard (2) = 35 endpoints, 20 dans Postman.

6 Installation & lancement

Pre-requis : JDK 21 (ms-21.0.12.1), Maven 3.9.16 (Toolbox), Node 22, PostgreSQL 18.

Backend

1. git clone <repo> && cd ShopFlow
2. JAVA_HOME=unknown mvn compile -> BUILD SUCCESS
3. Dev : mvn spring-boot:run -> http://localhost:8080/swagger-ui.html + /h2-console (jdbc:h2:mem:shopflow sa)
4. Prod : sudo systemctl start postgresql + DB_USERNAME=shopflow DB_PASSWORD=shopflow mvn spring-boot:run -Dspring-boot.run.profiles=prod
5. Tests rapides : curl -X POST /api/auth/register -d '{...}' -> accessToken

Frontend

1. cd frontend && echo "NEXT_PUBLIC_API_URL=http://localhost:8080" > .env.local
2. npm install && npm run dev -> http://localhost:3000
3. npm run build -> 11 routes static

Demo soutenance 3 min : 1. Register SELLER -> 2. POST /categories + POST /products (Seller) -> 3. Register CUSTOMER -> 4. Add to cart -> 5. POST /orders -> 6. GET /orders/my -> 7. PUT /cancel (si PENDING) -> 8. Dashboard admin.

Annexe - Swagger + Postman

Swagger UI accessible sur http://localhost:8080/swagger-ui.html via springdoc-openapi-starter-webmvc-ui 2.8.13 (Spec 3.1). Collection Postman : shopflow.postman_collection.json (20 requêtes, variables {{base}} + {{accessToken}}). Livable ZIP : backend/ (sans target/), frontend/ (sans node_modules/.next), shopflow.postman_collection.json, README.md, rapport.pdf.

Ressources : Spring Boot docs, Spring Security 6, Lombok, MapStruct, Next.js docs (Spec p.8).

****ShopFlow - yosrii - 22 Septembre 2026 - Git 3b7f7ba - 67 fichiers backend + 11 routes frontend - BUILD SUCCESS****